PART I. Divide and Conquer: Counting Inversions

1A(a). Brute-force implementation

BruteForce.java checks every pair (i,j) in an array where i < j and increments a counter whenever A[i] > A[j]. Two nested loops check each element against every element after it. For an array of length n, the total number of comparisons is (n-1) + (n-2) + ... + 1 = n(n-1)/2. Each comparison and counter increment takes constant time, giving Theta(n^2) runtime in the best, average, and worst cases. The implementation uses O(1) auxiliary space and a long counter.

1A(b). Divide-and-conquer implementation

The divide-and-conquer algorithm uses a modified Merge Sort to count inversions while sorting the array. It copies the input, divides the copy into two halves, and recursively counts inversions in each half. It then merges the sorted halves and counts cross inversions. When the current right-half element is smaller than the current left-half element, it is smaller than every remaining element in the sorted left half. The merge therefore adds mid - i + 1 to the count in one step.

Correctness explanation

Every inversion belongs to exactly one of three categories: left-half inversions, right-half inversions, or cross inversions. Recursive calls count the first two categories. During merging, selecting a smaller right-half element counts every inversion between it and the remaining left-half elements. Selecting a left-half element introduces no inversion with any remaining right-half element. Each cross inversion is counted when its right-half element is selected, so none is missed or counted twice.

The base case contains zero or one element, which is already sorted and has no inversions. Assuming the recursive calls correctly sort and count their halves, the merge produces a sorted combined array and counts all cross inversions. Adding the three counts gives the total. This establishes correctness by induction on the array size. The copied input also preserves the original array.

Complexity analysis

The recurrence is T(n) = 2T(n/2) + Theta(n), with constant-time base cases. There are two subproblems of half the size and linear merge work. By Case 2 of the Master Theorem, a = 2, b = 2, and f(n) = Theta(n), giving Theta(n log n) total runtime. Temporary merge arrays and the copied input require O(n) peak auxiliary space, with O(log n) recursion depth.

PART I. Validation and Runtime Scaling

1B. Validation and measurement method

Validation passed 12 hand-checkable arrays and 200 seeded random arrays of length 0-64 using seed 40502026. The hand cases include empty and singleton arrays, sorted and reverse-sorted inputs, nearly sorted and randomly ordered inputs, negative values, and integer extremes. Both counters must agree exactly. Independently derived hand counts also check the baseline. A library sort checks the optimized output for both sorted order and preservation of values. Both methods are checked for input mutation.

The final experiment uses uniformly shuffled permutations of distinct values 0 through n-1. Each size receives three warmup trials and seven measured trials. Both methods receive independent copies of the same input, and their execution order alternates. System.nanoTime measures complete solver calls, including internal allocations. Input generation, caller-side copying, output checking, and CSV writing are outside timing. Table 1 reports medians from this final run, replacing earlier measurements.

Array size n	Brute force median (ns)	Divide-and-conquer median (ns)
16	2,900	2,200
32	500	4,500
64	1,000	8,700
128	5,300	23,100
256	26,900	43,600
512	44,100	67,900
1,024	170,200	156,600
2,048	696,400	173,900
4,096	2,490,400	346,400

Table 1. Inversion runtime results from results/final-report.

In this run, divide and conquer is faster at n = 16, but brute force is faster from n = 32 through 512. Divide and conquer becomes consistently faster among the tested sizes at n = 1,024 and above. At n = 4,096, the medians are 2.4904 ms for brute force and 0.3464 ms for divide and conquer. Small-input timing noise and Java optimization affect the crossover, so these results do not establish a universal threshold. Larger sizes show the advantage expected from Theta(n log n) versus Theta(n^2). Processor speed, allocations, JVM optimization, and background activity can change the observed crossover.

PART II. Dynamic Programming: Coin Change

2A. Common interface and correctness

All four solvers implement CoinSolver and accept unique positive denominations and a nonnegative target A. They validate inputs, sort a copy of the denominations, and preserve the original array. They return a coin count and chosen list, or CoinResult.IMPOSSIBLE. Target zero returns an empty list. Let k be the number of denominations, m the smallest denomination, and d = floor(A/m).

For the exact solvers, every nonempty optimal solution has a final coin c. Removing that coin leaves an optimal solution for A-c; otherwise replacing the remainder with a better solution would improve the original. Thus best(A) is the minimum of 1 + best(A-c) over feasible final coins. The base value is best(0) = 0, and unreachable amounts remain impossible. Naive recursion explores this recurrence, memoization caches its values, and bottom-up DP evaluates smaller amounts first. Reconstruction follows recorded choices whose counts decrease toward the zero base case.

2A(a). Greedy

Greedy walks denominations from largest to smallest, repeatedly subtracting the current coin while it fits. If the remainder reaches zero it returns the chosen list. Otherwise it returns IMPOSSIBLE. This follows the largest-feasible-coin rule, but arbitrary coin systems can make its answer suboptimal or cause failure on a feasible target. Sorting costs O(k log k), and selection costs O(k + A/m). Auxiliary space is O(k), plus O(A/m) space for the output list.

2A(b). Naive recursion

NaiveRecursive tries every feasible final coin without caching results. Each recursive branch reduces the remaining amount by at least m, so depth is at most d + 1. Repeated subproblems cause exponential work. For k greater than 1, the recursion tree has an O(k^d) node bound. A conservative implementation bound including per-node loops and list handling is O(k log k + (k+d)k^d). For one denomination, recursion follows a single chain. Live partial lists retained across recursive frames can use O(d^2) space in a conservative worst-case bound, in addition to O(k) input storage and O(d) stack depth.

2A(c). Top-down memoization

Memoized uses the same recurrence but stores solved counts and reconstruction choices. Each reachable amount is solved once. Later requests reuse its cached result. Runtime is O(k log k + kA). Arrays and measured-state tracking use O(A) space, the copied denominations use O(k), and recursion depth is O(d). Total auxiliary space is O(k + A), excluding the returned list. Stack and call-budget limits can stop very large recursive runs.

2A(d). Bottom-up DP

BottomUp computes amounts 0 through A in increasing order. For each amount, it tries each usable denomination and stores the smallest feasible count and its chosen coin. Reconstruction follows these choices to zero. Ascending coin order and strict improvement retain the smaller final coin on a tie. Runtime is O(k log k + kA), and auxiliary space is O(k + A), including the sorted denomination copy, DP arrays, and measured-state set.

PART II. Exact-Solver Validation and Greedy Failure

2B. Hand-checkable and randomized cases

All three exact solvers agreed on the optimum or IMPOSSIBLE for the following 10 hand cases and 100 random instances. Each hand expectation is checked against all exact solvers. Different optimal coin lists are accepted when their counts agree.

Denominations	Target	Optimum	Hand explanation
{1,3,4}	0	0	Empty list
{2,4}	3	IMPOSSIBLE	Even coins cannot sum to 3
{5}	10	2	5+5; one coin is too small
{5}	3	IMPOSSIBLE	Smallest coin exceeds target
{1,3,4}	6	2	3+3; no coin equals 6
{1,2,3}	4	2	1+3 or 2+2; no coin equals 4
{4,1,3}	6	2	3+3; unsorted input
{1,5,10}	10	1	One 10 coin
{2,3}	7	3	2+2+3; two coins total at most 6
{3,4}	6	2	3+3; greedy fails after choosing 4

Table 2. Independently hand-checked outcomes.

Random cases use seed 40502026, one to four unique denominations, and target amounts from 0 to 12. Eighty cases include denomination 1. Every fifth case excludes 1 and draws denominations from 2 through 8, allowing unreachable targets. All 100 cases completed naive recursion. Every returned list, including greedy output, is independently checked for denomination membership, positive coins, matching count, and a sum equal to A. A long accumulator prevents sum overflow. The tests also check input preservation. Exact solvers establish impossibility by agreement and hand expectations; greedy may fail on a feasible target. Finite testing supports correctness on tested inputs.

Automated counterexample search

After exact-solver validation, the search enumerates all systems {1,a,b} with 2 <= a < b <= 8 and targets 1-12. These 252 additional instances are cross-checked with all three exact solvers. It found 14 feasible-but-suboptimal greedy cases across eight distinct systems, including the required sanity case.

Denominations	Target	Greedy solution	Optimal solution
{1,3,4}	6	4+1+1 (3 coins)	3+3 (2 coins)
{1,4,5}	8	5+1+1+1 (4 coins)	4+4 (2 coins)
{1,4,6}	8	6+1+1 (3 coins)	4+4 (2 coins)

Table 3. Three distinct greedy counterexamples. For {1,3,4} and A = 6, choosing 4 leaves remainder 2, requiring two 1 coins. Choosing 3 leaves another 3 and finishes in two coins. No single coin equals 6. The largest local choice therefore blocks the better pair of medium denominations.

PART II. Memoization and Repeated Subproblems

2C. Experiment 1: Repeated subproblems

We compared naive and memoized recursion using denominations {1,3,4}, extending targets from 0 through 30 in increments of three. Each run uses fresh metrics and, for memoization, a fresh cache. Recursive calls count every helper entry, including bases and cache lookups. Distinct states count unique target amounts completed. Cache hits count memoized calls that reuse an already solved amount. Naive recursion has zero cache hits.

Both algorithms use the same recurrence, but different coin orders reach the same remaining amount. Naive recursion recomputes those amounts, while memoization reuses their stored answers. At A = 18, naive recursion makes 7,920 calls while memoization makes 50, with 31 cache hits. Both complete 19 distinct target states, representing amounts 0 through 18.

Target	Naive calls	Memo calls	Naive states	Memo states	Cache hits	Naive status
0	1	1	1	1	0	OK
3	5	5	4	4	1	OK
6	24	14	7	7	7	OK
9	104	23	10	10	13	OK
12	441	32	13	13	19	OK
15	1,869	41	16	16	25	OK
18	7,920	50	19	19	31	OK
21	33,552	59	22	22	37	OK
24	142,129	68	25	25	43	OK
27	602,069	77	28	28	49	OK
30	1,000,001	86	29	31	55	CUTOFF

Table 4. Actual recursive work. Memoized runs all completed. The naive A = 30 row contains partial-work counters because execution was stopped.

The documented safety budget is 1,000,000 recursive calls per solve. The guard increments the counter at helper entry and stops when it exceeds the budget, so the cutoff row records 1,000,001 attempted calls. At A = 27, naive recursion completed with 602,069 calls. At A = 30, it exceeded the budget, while memoization completed in 86 calls with 55 cache hits and 31 distinct states. The cutoff defines the practical computation budget for this experiment. It does not mean the target is impossible or that the interrupted run found an optimum.

The naive A = 30 run had completed 29 distinct states before interruption. Its counters must not be compared as if it solved the whole instance. The rapid growth from 7,920 calls at A = 18 to 602,069 at A = 27, followed by the cutoff at 30, demonstrates the cost of overlapping recursive work. Memoization remains close to linear in A for this fixed coin set.

The CSV also records single-run elapsed times with reconstruction checks outside timing. Instrumentation is included in solver time. These single timings are descriptive, while call/state/cache counts are the main evidence in this experiment. All values, including the cutoff status, are retained in memoization.csv.

PART II. Pseudo-Polynomial Scaling and Reproducibility

2C. Experiment 2: Pseudo-polynomial scaling

We tested bottom-up DP with fixed denominations {1,3,4}, doubling A from 256 to 16,384. Each target receives three warmup runs and seven measured runs. A fresh metrics object is used each time. The timed region includes the complete solve and instrumentation, while output validation and CSV writing occur afterward. Table 5 reports medians from the final run.

Target A	Bit length	DP states	Median runtime (ms)
256	9	257	0.0096
512	10	513	0.0212
1,024	11	1,025	0.0413
2,048	12	2,049	0.0864
4,096	13	4,097	0.1683
8,192	14	8,193	0.2247
16,384	15	16,385	0.4972

Table 5. Fixed-k bottom-up measurements. DP state counts include the zero base case, giving A+1 states.

Doubling A approximately doubles the number of processed states, while its bit length increases by one. With k fixed, the O(kA) bound is linear in the numerical target. If b is the target bit length, A can be as large as 2^b - 1, so work linear in A can be exponential in b. This is why the algorithm is pseudo-polynomial. Runtime need not double exactly because allocation, JVM optimization, caching, and background activity affect measurements. State counts directly show the growth in evaluated amounts.

Reproduction and data

The project uses Java 21-compatible source and the Java standard library. The final measurements were collected on the same Windows machine with JDK 27. Compile from the repository root, then run the full validation and experiments:

```
javac --release 21 -d out "@sources.txt"
java -cp out common.Run all results/final-report
```

Configuration is recorded in src/common/ExperimentConfig.java. Seed 40502026 controls randomized validation and inversion inputs. Part 1 writes raw trials, median summaries, validation totals, and environment metadata in an inversions-* subdirectory. Part 2 writes part2b_validation.csv, greedy_counterexamples.csv, memoization.csv, pseudo_polynomial_raw.csv, and pseudo_polynomial_summary.csv. The report tables use the results/final-report run. Reruns may produce different timings and should use a new output directory to preserve prior measurements.

Measurement limits

The recursion call budget bounds naive work. The configured maximum scaling target bounds DP memory and execution time. Warmups reduce but do not eliminate JVM effects. Comparisons include each implementation's own allocations and reconstruction, so the results describe these complete implementations. Passing validation supports tested behavior, while the correctness arguments explain general algorithm behavior. Timing trends alone do not prove asymptotic bounds.

PART III. Cross-Paradigm Engineering Reflection

3(1). Work avoided by divide and conquer

Brute force checks n(n-1)/2 pairs, or 8,386,560 pairs at n = 4,096. The merge method uses sorted order to count several cross inversions with mid - i + 1. Its work is Theta(n log n), rather than quadratic. In the final run, the medians at n = 4,096 were 0.3464 ms for divide and conquer and 2.4904 ms for brute force.

3(2). Independent and overlapping subproblems

Inversion recursion splits distinct index ranges, so the two halves can be solved independently before merging. In coin change, different coin orders reach the same remaining target. At A = 18, naive recursion made 7,920 calls while completing only 19 distinct target states. These states are remaining amounts, not denominations.

3(3). The role of sorting

Sorted halves make it possible to infer that a right-half element smaller than the current left element is also smaller than every remaining left element. Those inversions are counted together. Each merge takes linear time and there are logarithmically many recursion levels, giving Theta(n log n). The optimization changes the asymptotic growth, rather than merely reducing the quadratic comparison count by a constant factor.

3(4). Same recurrence, different computation

Naive and memoized solvers evaluate the same final-coin recurrence. Naive recursion repeatedly expands the same targets, while memoization solves each reachable target once and reuses it. For fixed k, this reduces exponential repeated work to O(kA) recurrence work. At A = 30, naive recursion exceeded one million calls while memoization completed in 86.

3(5). Baselines and exact solvers as validation tools

The brute-force inversion counter supplied small-instance expected results for the optimized counter. Hand expectations also checked the baseline. For coin change, agreement among three exact solvers established a checked optimum before DP was used to find greedy failures. Independent reconstruction checks confirmed valid coin lists. Greedy used three coins for {1,3,4}, A = 6, while the exact optimum was two. These checks establish more than relative speed, though finite tests have limits.

3(6). Choosing an approach for a new problem

Divide and conquer is promising when a problem separates into smaller independent instances whose answers can be combined efficiently. Dynamic programming is promising when an optimal solution has optimal substructure and many branches revisit the same states. The inversion merge exploits independent halves, while coin-change memoization exploits repeated remaining amounts.

AI use and contributions

Our group used OpenAI ChatGPT and Codex for coding assistance, debugging, explanations, documentation, and interpretation of results. We reviewed generated code and tested the Java implementations. Codex also corrected the merge loop, moved validation outside experiment timing, extended the recursion experiment, reran validation, and revised this report using the resulting CSV files. AI output alone is not correctness evidence. The group remains responsible for reviewing the code, measurements, and final submission.

Jared worked on 1A(a), 1B, and 2B. Aimee worked on 1A(b) and 2C, including code and the written report. Fabian worked on 2A and Part III, including code and the written report.